

✅ 📅 DAY 2 — EMBEDDINGS + VECTOR RAG (UPDATED FOR YOU)

🎯 Goal

👉 Build **REAL RAG** using local embedding (Qwen)

✅ 🛠️ CHANGE (IMPORTANT — YOUR VERSION)

Instead of HuggingFace ❌ 👉 You used:

ollama.embeddings(model="qwen2.5") ✅ This is perfect for your restriction

✅ 📁 FINAL FILES (IMPORTANT)

📁 **embeddings/embedding_service.py**

✅ YOUR VERSION:

```
import ollama

class EmbeddingService:

    def encode(self, texts):

        if isinstance(texts, str):

            texts = [texts]

        embeddings = []

        for text in texts:

            response = ollama.embeddings(

                model="qwen2.5", prompt=text )

            embeddings.append(response["embedding"])

        return embeddings
```

 embeddings/vector_store.py

 No change

 retrieval/vector_search.py

 FINAL:

```
from embeddings.embedding_service import EmbeddingService
from embeddings.vector_store import VectorStore

class VectorSearch:

    def init(self):

        self.embedder = EmbeddingService()

        self.store = VectorStore()

    def index(self, documents):

        embeddings = self.embedder.encode(documents)

        ids = [str(i) for i in range(len(documents))]

        metadata = [{"source": "file"} for _ in documents]

        self.store.add(ids, documents, embeddings, metadata)

    def search(self, query):

        query_embedding = self.embedder.encode(query)[0]

        results = self.store.query(query_embedding)

        return results
```

 app/main.py (UPDATED)

```
from retrieval.vector_search import VectorSearch

from ingestion.loader import load_text_file

from models.llm import generate
```

```
def main():  
    print("Multimodal AI System (RAG Enabled)")  
    vs = VectorSearch()  
    filepath = "data/raw/sample.txt"  
    docs = load_text_file(filepath)  
    vs.index(docs)  
    print("Documents indexed!")  
    while True:  
        query = input(">> ")  
        if query.lower() == "exit":  
            break  
        results = vs.search(query)  
        context = "\n".join(results)  
        prompt = f"""  
Answer only from the context below.  
Context:  
{context}  
Question:  
{query}  
"""  
        response = generate(prompt)  
        print("\n", response)  
if name == "main":  
    main()
```

✅ OUTPUT NOW

Documents indexed!

>> what is ai

- ✅ Retrieval working
- ✅ Context being used
- ✅ Qwen answering

🧠 WHAT YOU HAVE BUILT (IMPORTANT)

👉 You now have:

Component	Status
LLM	✅ Qwen
Embedding	✅ Qwen embeddings
Vector DB	✅ Chroma
Retrieval	✅ Semantic search
Pipeline	✅ End-to-end

👉 This is:

✅✅ **REAL WORKING RAG SYSTEM (LOCAL)**

⚠️ LIMITATION (HONEST)

Using Qwen embeddings:

- ✅ works
- ❌ not best quality

👉 But perfect for your environment ✅

WHAT CHANGES FROM ORIGINAL PLAN

Removed:

- HuggingFace embeddings
- internet dependency

Added:

- Ollama embeddings
 - Fully local pipeline
-

CURRENT LEVEL (IMPORTANT)

You are now at:

LLM user 

Basic coder 

 RAG SYSTEM BUILDER 

NEXT STEP (DAY 3)

Now things get **serious improvement**

 We will add:

-  BM25 (keyword search)
 -  Hybrid retrieval (vector + keyword)
 -  RRF fusion (your original idea )
-